Decoy: Multi-Agent Environment for Hidden-Role Strategic Learning

Ayushi Bhattacharya

Abstract

Decoy is a compact multi-agent environment for studying hidden-role strategic learning under asymmetric information and noisy financial observations. Four agents interact in a general-sum game combining partial observability, Ornstein-Uhlenbeck market dynamics, and socially reactive behaviour. Under frozen-opponent training, tabular Q-learning reaches 98.4% overall Fraudster detection. Across the 500,000-episode joint-training runs, mean overall detection is 29.2%, alongside a 40.8% timeout rate. Among episodes ending in a terminal identification, the Fraudster identification rate reaches 49.3%. Joint training is also accompanied by longer episodes and more accusations. These experiments provide an inspectable tabular baseline for studying co-adaptation without deep function approximation.

1 Introduction

Hidden-role environments require agents to infer latent identities from incomplete and strategically manipulated evidence. Unlike standard classification, the evidence available to an agent is influenced by the actions of other agents, causing the effective learning problem to evolve as policies change. This creates a partially observable strategic setting in which detection, deception, and adaptation interact throughout an episode [3].

We model Decoy as a finite-horizon partially observable stochastic game with agents $\mathcal{G}$, latent state space $\mathcal{S}$, joint action space $\mathcal{A}$, transition dynamics, agent-specific observations, and reward functions $\mathcal{R}$ [5]. Trader roles are hidden from the Investigator and cannot be inferred from a fixed trader identity. Role-conditioned trader learners are selected externally, and the role label is excluded from each learner's state. Financial and social actions influence the evidence available throughout each episode. We evaluate tabular Q-learning and Monte Carlo learning against frozen opponents and Q-learning under joint training.

This paper contributes a compact financial and social hidden-role environment, inspectable tabular baselines across three training regimes, and empirical diagnostics linking the training-regime gap to timeouts, information gathering, accusations, and conditional identification accuracy.

2 Related Work

Serrino et al. [10] study hidden-role learning in *The Resistance: Avalon* using CFR with deep value networks and explicit belief tracking.

Aitchison et al. [1] introduce *Rescue the General*, a mixed cooperative-competitive hidden-role MARL environment, and evaluate an intrinsic reward for Bayesian belief manipulation.

Savin and Tsang [9] study social deduction through iterative voting in *Werewolf*, showing richer voting mechanics improve coordination under restricted communication.

Sarkar et al. [8] study social deduction in an *Among Us*-style environment using reinforcement learning with large language model agents and learned communication policies.

Bard et al. [2] introduce *Hanabi* as a cooperative imperfect-information benchmark for studying coordination, implicit communication, and ad hoc teamwork.

Phan et al. [7] study stochastic partial observability in cooperative MARL and introduce AERIAL alongside the MessySMAC benchmark to evaluate learning under noisy observations and initial states.

Papoudakis et al. [6] survey the non-stationarity that arises in independent MARL as concurrently learning agents change the effective transition dynamics faced by one another.

These studies span hidden-role inference, restricted and language-based communication, cooperative imperfect-information learning, stochastic partial observability, and non-stationarity in multi-agent learning. Decoy complements the existing environments through a compact financial and social setting and a tabular study of stationary and jointly adapting policies.

3 Environment Design

Each action can affect evidence available later in the episode. Financial actions transform noisy market observations, while social actions update behavioural histories and influence later responses. The Investigator observes neither the role assignment nor the latent market values.

3.1 Game Overview

Each episode consists of four agents: an Investigator and three Traders. The three trader roles, Innocent, Fraudster, and Trickster, are randomly assigned to trader identities at the beginning of every episode. This ensures that role identity cannot be inferred from a fixed player index.

Let the set of agents be

$$\mathcal{G} = \{I, T_1, T_2, T_3\}, \tag{1}$$

where I denotes the Investigator and T_j denotes the jth Trader, and $\mathcal{T} = \{T_1, T_2, T_3\}$ denotes the set of traders.

The Investigator's objective is to identify the Fraudster before exhausting a limited attention budget or reaching the episode horizon. The Investigator does not observe the hidden-role assignment directly. Decisions are made solely from noisy market observations, behavioural history, inspection summaries, and evolving social interactions.

Trader identities also provide no direct indication of the assigned role. The environment selects a separate role-conditioned learner according to the hidden-role allocation, and the role label is excluded from that learner's state. This structure allows the roles to follow different objectives without exposing the role through a fixed trader index.

Figure 1 summarizes the episode cycle.

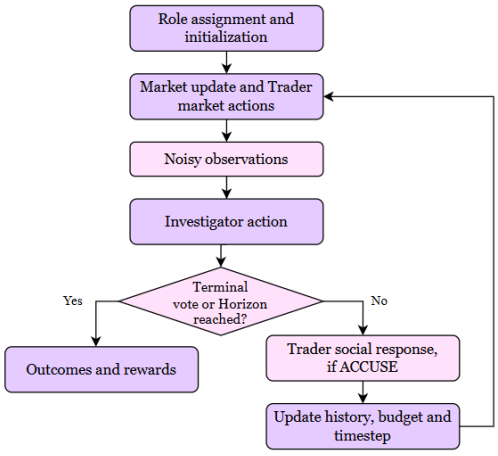


Figure 1: Episode interaction loop.

3.2 Stochastic Financial Dynamics

Independent Ornstein-Uhlenbeck processes drive the financial layer [12]. Each trader therefore has a latent market signal that evolves stochastically throughout the episode.

$$dX_t = \theta(\mu - X_t)\, dt + \sigma\, dW_t \tag{2}$$

The process was selected since it introduces continuous noisy variation while retaining mean-reverting behaviour controlled by θ. X_t denotes the latent market value, μ is the long-run mean, and σ determines the stochastic variation.

The implementation then advances the process using Euler-Maruyama discretization [4].

$$X_{t+\Delta t} = X_t + \theta(\mu - X_t)\Delta t + \sigma\sqrt{\Delta t}\,\epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0,1) \tag{3}$$

The environment initializes each trader with identical process parameters.

Table 1: Parameters of the Ornstein-Uhlenbeck process

Parameter	Value
θ	0.3
μ	0.0
σ	1.0
Δt	0.01

Each timestep begins by updating every trader's latent value before financial actions are applied. The resulting market observation is given by

$$Y_{i,t} = m(a^{\text{mkt}}_{i,t})X_{i,t} + \eta_{i,t}, \qquad \eta_{i,t} \sim \mathcal{N}(0, \sigma^2_{\text{obs}}) \tag{4}$$

The reported experiments use $\sigma_{\text{obs}} = 0.5$. A focused observation halves this standard deviation for the targeted trader during the next market update. The action-dependent modifier is

$$m(a) = \begin{cases} 1.0, & a = \text{NORMAL}, \\ 0.5, & a = \text{CONCEAL}, \\ 1.5, & a = \text{SIGNAL}. \end{cases} \tag{5}$$

3.3 Social Layer

The social layer uses role-dependent communication actions whose effects can propagate through later interactions. Behavioural history, accusations, and other behavioural factors influence subsequent decisions.

Social actions are represented as statement-target pairs, which are built from four statement types.

Each statement bundles the speaker, statement type, and an optional target, while the environment

continues to update the behavioural statistics, including accusation frequency, defence and support behaviour, and target switching.

Accusations trigger role-dependent reactions that influence later interactions.

3.4 State and Action Spaces

Diagnostic representations are separated from learning representations to allow detailed inspection while keeping tabular learning computationally manageable.

The Investigator maintains two state representations.

Table 2: Investigator state representations.

Representation	Components	Purpose
Diagnostic state	32	Inspection
Compact Q-state	16	Q-learning

The diagnostic representation retains detailed environment information. The compact state contains, for each trader, the current market bucket, inspection flag, and inspection-mean bucket. It also contains the latest statement and target, current accusation, budget and timestep buckets, remaining focus charges, and focus target.

The trader market and social learners each use a separate six-component learning state.

Table 3: Trader state representation.

Representation	Components
Trader market state	6
Trader social state	6

The market state contains the trader's local market value, the current accusation target, budget and timestep buckets, and the most recent social statement and target. The social state replaces the budget component with an indicator of whether the trader is currently accused. Both states exclude an explicit role label.

The action spaces are summarized below.

Table 4: Agent action spaces.

Agent	Action space
Investigator	OBSERVE, FOCUS, ACCUSE, VOTE
Trader (Market)	NORMAL, CONCEAL, SIGNAL
Trader (Social)	NEUTRAL, DEFEND, ACCUSE, SUPPORT

Let $\mathcal{A}_i$ denote the full action space of agent i and $\mathcal{A}_i^{\text{legal}}(s_i^t)$ the subset of actions valid under current state.

$$a_i^t \in \mathcal{A}_i^{\text{legal}}(s_i^t) \subseteq \mathcal{A}_i \tag{6}$$

Legal-action masking restricts action selection and value updates to valid actions during training and evaluation.

3.5 Role Objectives and Rewards

Investigator. The Investigator receives a noisy observation of a latent market signal generated by the Ornstein-Uhlenbeck process. Each trader's financial action modifies the underlying market value before it is observed through Gaussian noise, meaning the Investigator never has direct access to the hidden state. Instead, decisions must be made from imperfect evidence.

The Investigator operates under a finite attention budget. OBSERVE, ACCUSE, and VOTE each consume one unit. FOCUS uses one of two separate focus charges and reduces the observation noise for its target during the next market update.

The limited budget introduces an explicit trade-off between gathering additional evidence, recording an accusation, and committing to a terminal vote before the episode ends. Accusations are interactive and non-terminal. They can lead to financial and social reactions that alter the evidence available in subsequent timesteps, making investigation part of an evolving feedback loop.

The reward structure distinguishes between correct Fraudster identification, incorrect terminal identifications, and timeout:

$$R_I(o) = \begin{cases} +10, & o = \text{fraudster\_caught}, \\ -10, & o = \text{innocent\_caught}, \\ -3, & o = \text{trickster\_caught}, \\ -1, & o = \text{timeout}. \end{cases} \tag{7}$$

An `ACCUSE` action records the Investigator's current target and can be repeated. A terminal catch occurs only when the Investigator uses VOTE on the accused trader. The number of accusations therefore measures investigative behaviour, while Trader catches are terminal outcomes.

Innocent. Like the other traders, the Innocent can perform both financial and social actions. The environment selects its role-conditioned learner externally, while the learner's state excludes the role label and trader identity remains randomized across episodes. Trader rewards are separated into terminal outcome rewards and social-action rewards. Terminal rewards are assigned when an episode ends through a vote or timeout, and social rewards are recorded as separate transitions. The objective is to survive the investigation and avoid suspicion.

The Innocent receives the terminal reward

$$R_{\text{Innocent}}(o) = \begin{cases} +5, & o = \text{Fraudster caught}, \\ -10, & o = \text{Innocent caught}, \\ +1, & o = \text{Trickster caught}, \\ 0, & o = \text{Timeout}, \end{cases}$$

In the reported configuration, accusations receive 0 and the remaining legal non-neutral social actions receive -1.

Fraudster. The Fraudster shares the same financial and social action spaces as the other traders but operates under a different objective. Its incentives favour avoiding capture, concealing suspicious behaviour, and redirecting terminal identification toward another trader.

The Fraudster receives the terminal reward

$$R_{\text{Fraudster}}(o) = \begin{cases} -10, & o = \text{Fraudster caught}, \\ +10, & o = \text{Innocent caught}, \\ +2, & o = \text{Trickster caught}, \\ +1, & o = \text{Timeout}, \end{cases}$$

The $+2$ reward for a Trickster catch differs from the Investigator's -3 reward for the same outcome, contributing to the general-sum structure.

In the reported configuration, accusations receive $+1$ and the remaining legal social actions receive 0.

Trickster. The Trickster combines learned market and social Q-policies with an adaptive WinRegister. Before commitment, its learned social policy selects from the full legal action set. As training progresses, the WinRegister estimates which side is more likely to succeed. After commitment, the legal social actions are restricted to defending or supporting the predicted ally and accusing the predicted opponent.

For social actions, the Trickster receives $+1$ for ACCUSE and SUPPORT and 0 for NEUTRAL and DEFEND actions. Before commitment, its terminal shaping assigns $+1$ for either a Fraudster or Innocent catch and -1 if it was accused during the episode. Every 100 training episodes, it also adds twice the mean binary entropy over up to the 100 most recent decisive-history updates. After commitment, terminal shaping gives $+5$ when its chosen side wins, -5 when the other side wins or the Trickster is caught, and -1 if it was accused. Timeouts produce no side-specific win reward.

The estimated Investigator-side probability is

$$p_I = (1-\lambda)p_{\text{all}} + \lambda p_{\text{recent}}, \qquad \lambda = 0.65 \tag{8}$$

where p_{all} and p_{recent} denote the all-history and recent Investigator win rates, respectively. The recent rate uses up to the 500 most recent decisive outcomes. Fraudster catches are recorded as Investigator wins, Innocent catches are recorded as criminal-side wins, and timeouts and Trickster catches do not update either side's win history. The opposing side probability is

$$p_C = 1 - p_I. \tag{9}$$

The predicted winning side is then

$$\hat{w} = \arg\max_{w \in \{I,C\}} p_w, \tag{10}$$

Commitment occurs only when the minimum history, confidence, and expected-margin conditions are satisfied.

$$N_{\text{seen}} \geq N_{\min} \tag{11a}$$

$$p_{\hat{w}} \geq p_{\min} \tag{11b}$$

$$|p_{\hat{w}} - 0.5|\, N_{\text{rem}} \geq M_{\min} \tag{11c}$$

Here, N_{seen} is the number of completed training episodes recorded by the WinRegister, and $N_{\text{rem}} = \max(N_{\text{total}} - n - 1, 0)$ is the number of episodes remaining after the current zero-indexed episode n in a run of N_{total} episodes. The thresholds are $N_{\min} = 2000$, $p_{\min} = 0.60$, and $M_{\min} = 10$. This delayed commitment prevents the Trickster from following one side before enough outcome history is available.

4 Experimental Setup

The experiments use tabular reinforcement-learning methods to isolate strategic interaction and co-adaptation without neural function approximation. Q-learning is used as an online temporal-difference method [13], while Monte Carlo uses episodic returns [11]. Both maintain explicit state-action value tables, providing inspectable baselines for later function-approximation experiments.

4.1 Algorithms

The Investigator was trained using both Q-learning and Monte Carlo learning methods in the frozen-opponent setting. Q-learning updates value estimates using bootstrapped targets, and Monte Carlo updates them from complete episode returns. The two methods provide interpretable tabular baselines while retaining the same state and action representations. In the multi-agent setting, Q-learning was used for the Investigator and the market and social policies of each trader role. The Trickster's WinRegister additionally restricts its legal social actions after commitment.

For Q-learning, define the legal continuation value

$$V_Q(s) = \max_{a' \in \mathcal{A}^{\text{legal}}(s)} Q(s, a').$$

The target is

$$y_t = \begin{cases} r_{t+1}, & \text{at terminal transitions,} \\ r_{t+1} + \gamma V_Q(s_{t+1}), & \text{otherwise.} \end{cases}$$

The update is

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[y_t - Q(s_t, a_t) \right]. \quad (12)$$

Monte Carlo uses the discounted episode return

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_{k+1} \quad (13)$$

and performs an every-visit sample-average update after an episode ends:

$$N(s_t, a_t) \leftarrow N(s_t, a_t) + 1, \quad (14)$$

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \frac{G_t - Q(s_t, a_t)}{N(s_t, a_t)}. \quad (15)$$

4.2 Training configuration

The main training budgets ranged from $10,000$ to $500,000$ episodes, with three independent random seeds used for each reported condition. The frozen-opponent sweeps used seeds $\{11, 42, 99\}$, and the joint-training sweeps used $\{71, 74, 77\}$. Episodes were limited to 20 timesteps with an Investigator attention budget of 10. Q-learning used $\alpha = 0.1$ and $\gamma = 0.99$, with fixed $\varepsilon = 0.1$ exploration during training. Evaluation used greedy policies with $\varepsilon = 0$. Legal-action masking was applied during training and evaluation so that value selection and bootstrapping considered only valid actions.

An additional adaptive learning-rate schedule was evaluated for the Investigator. After an initial 200-episode history, the mean reward over the most recent 100 episodes was compared with that of the preceding 100 episodes. A decrease exceeding 0.1 increased α by 0.005, while an increase exceeding 0.1 reduced α by the same amount, with α bounded between 0.03 and 0.12. This ablation tests whether recent performance can guide the learning rate under evolving opponent behaviour. It was evaluated at the $1,000$-episode budget.

4.3 Evaluation Regimes

The frozen-opponent and joint-training regimes form the main sweeps. Staged training and cross-evaluation are used as $10,000$-episode diagnostics. Table 5 summarizes the five evaluation regimes.

Table 5: Evaluation regimes used to study stationary learning, co-adaptation, and transfer.

Regime	**What it tests**
Frozen $\mathcal{T}$, learned I	Stationary Investigator learning
Learned $\mathcal{T}$ vs. frozen I	Trader role differentiation
Joint training of $\mathcal{G}$	Concurrent policy adaptation
Frozen I vs. joint-trained $\mathcal{T}$	Joint-policy transfer
Cross-evaluation	Transfer across training regimes

Table 6 reports the frozen-$\mathcal{T}$, learned-I regime. Table 7 evaluates staged and jointly trained Traders against a frozen Investigator. Table 8 and Figure 3 report the main joint-training sweep, while Table 9 evaluates staged and jointly trained Traders against jointly trained Investigators.

Frozen-opponent experiments isolated learning against stationary behaviour, while joint training exposes I and $\mathcal{T}$ to changing opponent policies. Cross-evaluation measures transfer across training conditions.

4.4 Metrics

Frozen-opponent and cross-evaluation policies were evaluated over $2,000$ greedy episodes per seed. The main joint-training sweep reports training-time summaries and rolling trajectories from each complete run. Metrics include mean reward, role-specific detection rates, timeout rate, episode length, remaining attention budget, and learned-state count. Behavioural analysis additionally considers observations, accusations, and votes. Frozen-opponent results are reported as mean $\pm$ standard deviation across three seeds. Joint-training and cross-evaluation tables report three-seed means.

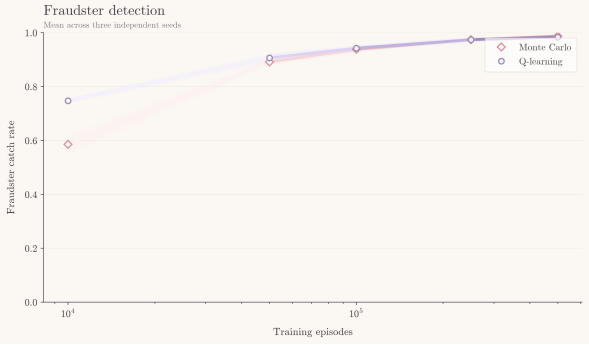
5 Results and Discussion

The experiments show a distinction between learning against stationary opponents and learning under co-adaptation. In the frozen-opponent setting, both tabular methods eventually achieve high Investigator performance, with Q-learning showing a stronger early advantage. The joint-training histories show substantially lower overall Fraudster detection, a higher timeout rate, and a different investigative action pattern.

5.1 Frozen-opponent convergence

Under frozen-opponent training, Q-learning reaches 74.8% Fraudster detection after 10,000 episodes, compared to Monte Carlo, which reaches 58.6%. At 500,000 episodes, Q-learning reaches 98.4% detection while Monte Carlo reaches 98.7%. Q-learning also achieves higher evaluation reward at a lower budget, while the gap narrows as training increases.

(a) Mean evaluation reward.



(b) Fraudster detection rate.

Figure 2: Frozen-opponent learning across training budgets for tabular Q-learning and Monte Carlo. Shaded regions show one standard deviation across three seeds.

The observed early advantage for Q-learning is consistent with its use of bootstrapped updates, which propagate value estimates before complete episodic returns are available. Monte Carlo updates from complete episodic returns and shows slower early improvement under the current configuration. Figure 2 shows the resulting trajectories across five training budgets. Monte Carlo progressively closes the gap with extended training. Within this frozen-opponent experiment, the methods differ most clearly in early learning efficiency and reach comparable final detection rates.

We refer to the Traders trained against a frozen Investigator as *staged Traders*.

As an additional diagnostic, roles were reversed by training the Traders against a frozen Investigator for 10,000 episodes. The resulting staged Trader policies were evaluated alongside jointly trained Trader policies against the same Investigator.

Table 6: Frozen-opponent performance at early and extended training budgets. Values are mean ± standard deviation across three seeds.

	10k episodes		500k episodes	
Algorithm	Detection	Reward	Detection	Reward
Q-learning	$74.8 \pm 0.9\%$	6.50 ± 0.10	$98.4 \pm 0.3\%$	9.81 ± 0.04
Monte Carlo	$58.6 \pm 3.4\%$	4.20 ± 0.47	$98.7 \pm 0.1\%$	9.84 ± 0.01

Table 7: Frozen-investigator cross-evaluation at 10,000 training episodes. Values are means across three seeds.

Condition	Fraudster	Innocent	Trickster	Timeout	Reward
Frozen I / staged $\mathcal{T}$	24.0%	24.2%	26.2%	25.8%	−1.058
Frozen I / joint $\mathcal{T}$	28.8%	25.5%	29.1%	16.5%	−0.709

5.2 Joint-training dynamics

Across the joint-training histories, overall Fraudster detection remains near 25% through 250,000 episodes and reaches 29.2% in the 500,000-episode runs. These rates include timeout episodes and are far below the 98.4% – 98.7% overall detection achieved in frozen-opponent evaluation. The three-role 33.3% chance comparison applies only to episodes ending in a terminal identification. The corresponding conditional rate is reported in Section 5.3. Mean training-time Investigator reward increases from −0.45 at 50,000 episodes to 0.63 at 500,000 episodes. This increase reflects the full terminal-outcome distribution, including fewer costly false catches and a larger share of timeouts.

The transition dynamics encountered by I change as the traders adapt, creating a non-stationary learning problem for fixed state-action values. The current experiments cannot determine how much of the gap comes from changing opponents and how much comes from reward asymmetry and increased timeouts.

Table 8: Training-time joint-policy summaries across training budgets. Values are means across three seeds.

Episodes	Reward	Fraudster	Innocent	Trickster	Timeout
50k	−0.448	26.0%	20.9%	21.4%	31.7%
100k	−0.326	25.5%	19.3%	19.9%	35.3%
250k	−0.082	25.2%	16.6%	17.8%	40.4%
500k	0.629	29.2%	14.1%	15.9%	40.8%

From 50,000 to 500,000 training episodes, Trickster detection decreases from 21.4% to 15.9%, and Innocent detection decreases from 20.9% to 14.1%. Timeout frequency increases from 31.7% to 40.8%. The outcome distribution therefore shifts away from incorrect terminal catches and toward timeouts.

A similar 10,000-episode cross-evaluation was

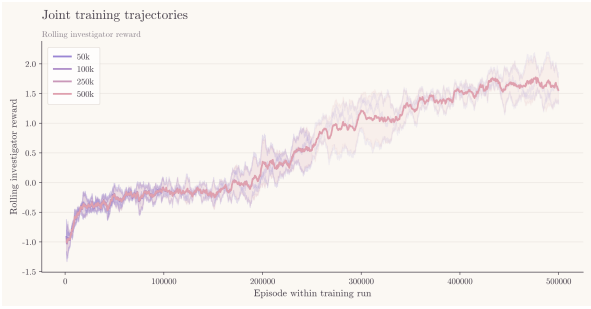


Figure 3: Joint-training reward trajectories. Shaded regions show one standard deviation across three seeds.

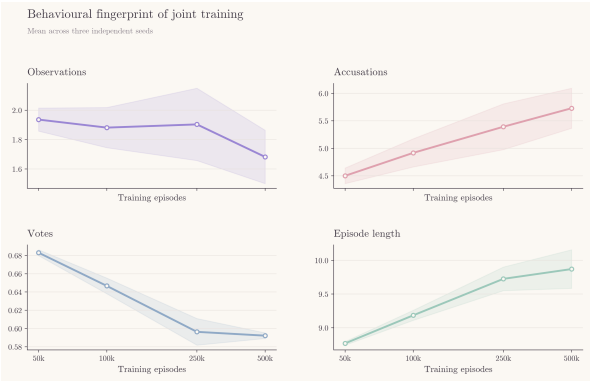


Figure 4: Behavioural metrics across joint-training budgets. Shaded regions show one standard deviation across three seeds.

used to compare jointly trained and staged Trader policies against jointly trained Investigators.

Table 9: Joint-policy diagnostics cross-evaluation at 10,000 training episodes. Values are means across three seeds.

Condition	Fraudster	Innocent	Trickster	Timeout	Reward
Joint I / staged $\mathcal{T}$	24.7%	24.7%	24.8%	25.8%	−1.002
Joint I / joint $\mathcal{T}$	27.0%	23.2%	22.1%	27.8%	−0.558

At the 10,000-episode budget, jointly trained Trader policies are slightly easier to detect in both cross-evaluations. Since the tables report three-seed means without dispersion estimates, these differences are treated as descriptive. The diagnostic also covers a shorter budget than the main joint-training sweep.

5.3 Behavioural analysis

The training-time behavioural metrics show a corresponding change in the Investigator's decision pattern from 50,000 to 500,000 episodes. Mean observations decrease from 2.03 to 1.68 per episode, while mean accusations increase from 4.50 to 5.73. Mean episode length increases from 8.77 to 9.87 timesteps, while timeout rate rises from 31.7% to 40.8%. Remaining attention budget decreases from 2.88 to 2.00.

The learned policy becomes more accusation-heavy while gathering less information. The increase in timeouts and episode length accompanies the persistent low overall Fraudster detection. These changes show that the joint-training performance gap is associated with a broader shift in the Investigator's behaviour. Figure 4 summarizes these training-time behavioural trends.

As another way to diagnose the joint-training histories, we calculate the conditional Fraudster identification rate specifically for non-timeout episodes:

$$P(F \mid \text{no timeout}) = \frac{N_F}{N_F + N_I + N_T}, \qquad (16)$$

where F denotes a Fraudster catch, and N_F, N_I, N_T denote the number of Fraudster, Innocent, and Trickster catches among non-timeout episodes. This rate increases from 38.1% at 50,000 training episodes to 49.3% at 500,000 episodes. The latter value exceeds the 33.3% chance level for a uniform choice among three traders. Terminal identification becomes more frequently correct among non-timeout episodes, while the increasing timeout frequency keeps overall Fraudster detection at 29.2%. The conditional and overall rates therefore describe separate aspects of the learned policy.

5.4 Adaptive-alpha null result

The adaptive learning-rate variant produced no change in the reported episode-outcome metrics at 1,000 training episodes. Across all three seeds, reward, role-specific detection, timeout rate, and behavioural metrics numerically matched the fixed-α condition, while the adaptive schedule produced different α values in two of three runs. This limited null result applies to one short training budget and does not establish the schedule's behaviour under longer training or additional seeds.

5.5 Trickster behaviour

The Trickster's WinRegister becomes increasingly confident in the Investigator side as joint training progresses. From 50,000 to 500,000 episodes, the all-history and recent-window Investigator win probabilities increase from 55.4% to 67.5% and 58.1% to 75.2%, respectively, while prediction confidence rises from 57.1% to 72.5%.

The WinRegister tracks the changing outcome distribution without explicitly modelling the Investigator's internal state. Figure 5 summarizes the resulting probability and confidence trajectories.

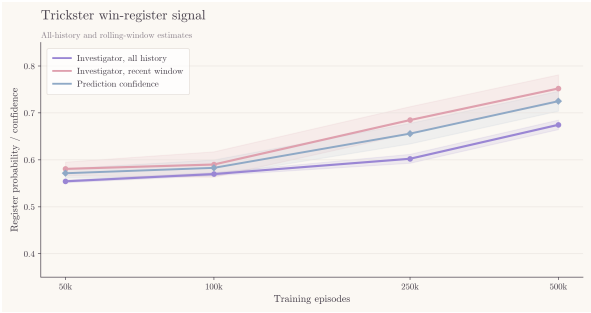


Figure 5: Trickster WinRegister dynamics across training budgets. Shaded regions show one standard deviation across three seeds.

6 Limitations

Decoy is intentionally scoped to a small, tabular multi-agent setting, which limits the extent to which the observed learning dynamics can be generalized to larger populations or continuous state spaces. The four-agent environment and hand-designed state representations make the learned policies directly inspectable, but different state abstractions or reward structures may produce different behaviours.

- **Statistical coverage**: The experiments use three random seeds, limiting the precision of variance estimates.
- **Learning structure**: The learning agents are independent, so the joint-training setting remains subject to non-stationarity.
- **Causal scope**: The comparison does not isolate non-stationarity, reward asymmetry, and timeout behaviour as separate causes of the training-regime gap.
- **Outcome interpretation**: Overall Fraudster detection includes timeouts, while the three-role chance baseline applies to terminal identifications. Both metrics are required to characterize the learned policy.
- **Reward sensitivity**: The timeout penalty is smaller than the penalties for incorrect terminal identifications, which can favour conservative voting. The current experiments do not include a reward-structure ablation.
- **Trickster model**: The WinRegister is heuristic and has no explicit belief model.
- **Adaptive learning-rate ablation**: The adaptive α ablation was evaluated only at $1,000$ episodes, leaving its behaviour at the longer training budgets untested.
- **Cross-evaluation**: The diagnostics were conducted at $10,000$ training episodes and therefore do not establish whether the transfer patterns persist at longer training budgets.

7 Future Work

Future work can extend Decoy across several directions. Reward-structure ablations can measure the sensitivity of timeout behaviour to terminal incentives. Training a fresh Investigator against frozen final checkpoints of jointly trained Traders can test whether the final policies remain learnable under stationary conditions. Deep function approximation can extend the current tabular representations to larger and continuous observation spaces. Opponent-aware and centralized-training methods can test whether explicit adaptation to changing trader policies reduces the joint-training gap. The WinRegister can also be replaced by an explicit Bayesian or recurrent opponent model. Larger populations and richer communication can test whether the observed behavioural patterns persist under increased strategic complexity.

8 Conclusion

Decoy provides a controlled setting for studying hidden-role strategic learning under partial observability and co-adaptation. Tabular learning achieves near-perfect overall Fraudster detection against stationary opponents. Joint-training histories exhibit substantially lower overall detection, a higher timeout rate, and improved conditional accuracy among terminal identifications. These results establish a clear training-regime gap and document the accompanying behavioural shift. Additional controls are required to separate the effects of opponent non-stationarity and reward-driven timeout behaviour. Decoy offers a compact baseline for these investigations in hidden-role multi-agent learning.

Code availability. The implementation and experiment scripts are available at `https://github.com/aiyu221b3/decoy_exp`.

References

[1] Matthew Aitchison, Lyndon Benke, and Penny Sweetser. Learning to deceive in multi-agent hidden role games, 2022. arXiv:2209.01551.

[2] Nolan Bard, Jakob N. Foerster, Sarath Chandar, Neil Burch, Marc Lanctot, H. Francis Song, Emilio Parisotto, Vincent Dumoulin, Subhodeep Moitra, Edward Hughes, Iain Dunning, Shibl Mourad, Hugo Larochelle, Marc G.

Bellemare, and Michael Bowling. The hanabi challenge: A new frontier for ai research. *Artificial Intelligence*, 280:103216, 2020.

[3] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.

[4] Peter E. Kloeden and Eckhard Platen. *Numerical Solution of Stochastic Differential Equations*. Springer-Verlag, Berlin, 1992.

[5] Michael L. Littman. Markov games as a framework for multi-agent reinforcement learning. In *Proceedings of the Eleventh International Conference on Machine Learning*, pages 157–163. Morgan Kaufmann, 1994.

[6] Georgios Papoudakis, Filippos Christianos, Arrasy Rahman, and Stefano V. Albrecht. Dealing with non-stationarity in multi-agent deep reinforcement learning, 2019. arXiv:1906.04737.

[7] Thomy Phan, Fabian Ritz, Philipp Altmann, Maximilian Zorn, Jonas Nüßlein, Michael Kölle, Thomas Gabor, and Claudia Linnhoff-Popien. Attention-based recurrence for multi-agent reinforcement learning under stochastic partial observability. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 27840–27853. PMLR, 2023.

[8] Bidipta Sarkar, Warren Xia, C. Karen Liu, and Dorsa Sadigh. Training language models for social deduction with multi-agent reinforcement learning. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2025)*, pages 1830–1839, Detroit, MI, USA, 2025. International Foundation for Autonomous Agents and Multiagent Systems.

[9] George Savin and Alan Tsang. Learning to vote differently in social deduction games. In *Proceedings of the 1st Workshop on Social Choice and Learning Algorithms (SCaLA 2024), at the 23rd International Conference on Autonomous Agents and Multiagent Systems*, Auckland, New Zealand, 2024.

[10] Jack Serrino, Max Kleiman-Weiner, David C. Parkes, and Joshua B. Tenenbaum. Finding friend and foe in multi-agent games. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[11] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.

[12] George E. Uhlenbeck and Leonard S. Ornstein. On the theory of the brownian motion. *Physical Review*, 36(5):823–841, 1930.

[13] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.